UML Diagram Description

Classes and Methods	Description
login()	Takes a username, checks if it is an existing user, and starts a session if it is. Users must not be logged in to log in.
IsLoggedIn()	Checks if users are currently logged in
startSession(username)	Starts a session with the account username that's been logged into
Session	Represents a session of using the game distribution system. Keeps track of session information such as the amount of credits added during that session.
getCurrentUser()	Gets the current user, so that information and transactions are displayed and saved for the correct user
isActive()	Checks and marks if a session is currently active
addSessionCredit()	Stores the amount of credits that's been added during the current session to prevent exceeding the limit
UserAccount	Represents the attributes of user account, and their permissions within the system. This contains information of if a user is actively logged in, their username, account type, amount of credits, and their owned games.
canBuy()	Sets and checks a user's permission to buy games
canSell()	Sets and checks a user's permission to sell games.
canAddCredit()	Sets and checks a user's permission to add credit.
canCreate()	Sets and checks a user's permission to create users.
canDelete()	Sets and checks a user's permission to delete users
canRefund()	Sets and checks a user's permission to refund users

isAdmin()	Sets and checks if a user is admin to easily set their permissions
getUserType()	Gets the type of a user to set their permissions and display the type accordingly
getUsername()	Gets the username of a user to display the username, and make updates to the correct user
getBalance()	Gets the balance of a user to display and make the correct updates to balance.
getOwnedGames()	Gets the games owned by a user
UserAccountsFile	Stores all the existing user accounts and their information
findUser(username)	Retrieves a user for signing in or performing transactions onto the user
userExists(username)	Checks if a user exists.
addCredit(amount)	Adds credit to user accounts' balance to either the current user or the specified username, depending if an admin performed the transaction.
updateCredit(username, amount)	Updates the balance of users
getSessionCredits()	Gets the amount of credits that have been added in the current session to not exceed the limit.
logAddCredit()	Logs the transaction for added credits
AdminUser	Represents users that are admin type (AA), and their permissions. This user class is able to buy, sell, refund, create users, delete users, add credits to users, and logout.
FullStandardUser	Represents users that are full standard type (FS), and their permissions. This user class is only able to buy, sell, add credits to themselves, and logout.
SellStandardUser	Represents users that are sell standard type (SS), and their permissions. This user class is only able to sell, add credits to themselves, and logout.

BuyStandardUser	Represents users that are buy standard type (BS), and their permissions. This user class is only able to buy, add credits to themselves, and logout.
create	Creates user accounts, which can only be done by Admin Users
addUser(username, type)	Adds new users to the user accounts file
userAlreadyExists(username)	Checks if a user already exists with a username when creating an account
logCreate()	Logs the transaction for a created account
delete	Deletes user accounts, which can only be done by admin users
removeUser(username)	Removes deleted users from the user accounts file
isCurrent()	Checks if the user being deleted is the current account to prevent it
logDelete()	Logs the transaction for deleting an account
refund	Refund credits to users, which updates the balance of users receiving refunds, and the sellers whose games are being refunded
logRefund	Logs a refund transaction
sell	Publishes a game up for sale with the desired price
hasPermission(type)	Checks the type of user using this transaction and if they have permission to sell before proceeding
gameAlreadyExists(game)	Checks if a game already exists with the username of the game trying to be sold
addgame(game, seller, price)	Adds games that are newly up for sale to the available games
listaccounts	Lists user accounts with their type and credits
logSell()	Logs the transaction for the sale of a game
buy	Buys games from a seller
addToLibrary()	Adds purchased games to the user’s owned

	games and saves it in the current games file
getPrice()	Gets the price of a game
logBuy()	Logs the transaction of buying a game
AvaliableGamesFile	Stores the available games for sale
findGame(game)	Finds desired game in AvaliableGameFile to perform transactions on it such as seeing if it exists to buy it, delete it, or sell it.
GameCatalogPage	Displays the list of available games using the Available Games File
GameLibrary	Displays the list of games from the game collection file.
GamesCollectionFile	Displays the games owned by users
ownsGame(username, game)	Checks if a user owns a specific game
getOwnedGames(username)	Gets all the owned games from a specific user
Game	Represents a game
getName()	Gets the name of a game
gerSeller()	Gets the seller of a game
logout	Logs a user out, and ends the session
endSession()	Ends a session of use of the game distribution system
logSession()	Logs the transaction of the end of the session
readFile()	Reads the contents of a file for data retrieval, and seeing things such as users, or games
writeFile()	Modifies files to add new data or remove existing data, such as users or games.
DailyTransactionFile	Stores the transactions that happened in a day
addTransaction()	Adds a transaction to the file
clearTransactions()	Clears the transactions in the file